

Diploma in Computer Science & Information Technology

SBTE, Bihar — Semester IV

Computer Organization and Architecture

Course Code: 2418404

Unit-wise Detailed Notes

UNIT 1: Basic Computer Organization and Design

1.1 Stored Program Organization

In a stored-program computer (Von Neumann architecture), both instructions and data are stored in the same memory. The CPU fetches instructions from memory one at a time and executes them. This architecture consists of: CPU (Control Unit + ALU), Memory, and I/O devices connected by buses.

1.2 Instruction Set and Instruction Cycle

An instruction set is the complete set of operations a CPU can perform. Each instruction specifies: operation code (opcode) and operand address(es).

Instruction Cycle (Fetch-Decode-Execute)

- **Fetch:** CPU reads the instruction from memory at the address in the Program Counter (PC). PC is incremented to point to next instruction.
- **Decode:** Control unit interprets the opcode to determine the operation and addressing mode.
- **Execute:** ALU or other hardware performs the specified operation.

1.3 Buses in Computer Systems

- **Address Bus:** Carries memory addresses from CPU to memory/I/O. Unidirectional. Width determines addressable memory space.
- **Data Bus:** Carries actual data between CPU, memory, and I/O. Bidirectional. Width determines how much data can be transferred at once.
- **Control Bus:** Carries control signals (read/write, interrupt, clock) from control unit. Manages timing and coordination.

Bus Arbitration

When multiple devices need the bus simultaneously, arbitration decides who gets access. Methods: Daisy chaining, Centralized arbitration, Distributed arbitration.

1.4 Registers and Stack Organization

Registers are small, fast storage locations inside the CPU. Common registers: Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Buffer Register (MBR), Accumulator (AC).

Register Stack: A set of registers organized as a LIFO stack using a Stack Pointer (SP). Memory Stack: Section of RAM used as a stack for function calls, local variables, and return addresses.

1.5 Addressing Modes

- **Immediate:** Operand is the data itself. Fast but limited size.
- **Direct:** Address in instruction is the effective address of operand in memory.
- **Indirect:** Address in instruction points to a memory location that contains the effective address.
- **Register Direct:** Operand is in a register. Very fast.
- **Register Indirect:** Register holds the address of the operand.

- **Indexed:** Effective address = base address + index register value. Used for array access.

1.6 Interrupts

An interrupt is a signal that temporarily suspends the CPU's current activity to handle an urgent event. After handling, execution resumes where it was interrupted.

- **Hardware Interrupts:** Generated by hardware devices (keyboard, timer, disk I/O completion).
- **Software Interrupts:** Generated by software instructions (system calls, exceptions).
- **Maskable Interrupts:** Can be enabled/disabled by setting a flag in the CPU status register.
- **Non-Maskable Interrupts (NMI):** Always processed immediately regardless of flag settings. Used for critical events like power failure.

1.7 Subroutine Call and Return

A subroutine (function) is a reusable code block. CALL instruction: saves the return address (PC value) on the stack and jumps to the subroutine. RETURN instruction: pops the return address from the stack and jumps back. The stack-based mechanism enables nested and recursive calls.

UNIT 2: Arithmetic and Logic Unit (ALU)

2.1 Signed Number Representation

Signed Magnitude

The MSB (most significant bit) represents the sign: 0 = positive, 1 = negative. Remaining bits represent the magnitude. Problem: Two representations of zero (+0 and -0). Addition/subtraction requires checking signs.

2's Complement Representation

Most common representation in modern computers. To negate: invert all bits and add 1. Only one representation of zero. Range for n bits: $-2^{(n-1)}$ to $2^{(n-1)}-1$. Addition and subtraction use the same hardware.

2.2 Multiplication Algorithms

Booth's Algorithm

An efficient algorithm for multiplying signed binary numbers in 2's complement. It handles both positive and negative numbers uniformly. Works by examining pairs of bits in the multiplier, performing add, subtract, or shift operations. Reduces the number of additions needed compared to simple binary multiplication.

Array Multiplier

A combinational circuit for fast parallel multiplication. Uses an array of AND gates and adders. All partial products are computed simultaneously. Faster than sequential multiplication but requires more hardware.

2.3 Division Algorithms

Binary division is performed by the hardware using a series of shift and subtract operations, analogous to long division. Signed magnitude division handles sign separately.

Divide Overflow: Occurs when the quotient exceeds the register capacity. The hardware must detect this condition and signal an error (overflow interrupt).

2.4 Floating-Point Arithmetic

Floating-point numbers represent real numbers with a fractional part. Format: sign bit, exponent (biased), mantissa (significand).

IEEE 754 Standard

- **Single Precision (32-bit):** 1 sign + 8 exponent + 23 mantissa bits.
- **Double Precision (64-bit):** 1 sign + 11 exponent + 52 mantissa bits.

IEEE 754 defines special values: +/-Infinity, NaN (Not a Number), and denormalized numbers for very small values.

Floating-Point Addition/Subtraction

Steps: (1) Align exponents (shift mantissa of smaller number right). (2) Add/subtract mantissas. (3) Normalize result (adjust exponent and mantissa). (4) Round result.

UNIT 3: Input-Output Organization

3.1 I/O Interface

The I/O interface connects peripheral devices to the CPU and memory via buses. It provides: data buffering, signal conversion, status/control registers, and data format conversion. Each peripheral has an interface module that bridges the device speed/format gap with the fast CPU.

I/O Bus vs Memory Bus

- Memory bus is faster (optimized for main memory speed); I/O bus is slower (designed for diverse peripherals).
- Memory bus addresses RAM; I/O bus addresses device registers.
- In some architectures (x86) I/O addresses are separate from memory addresses (port-mapped I/O); in others they share the address space (memory-mapped I/O).

3.2 Modes of Data Transfer

Programmed I/O (Polling)

CPU directly controls data transfer. CPU continuously checks (polls) the device status register to see if I/O is complete. Very simple but wastes CPU time — CPU cannot do other work while waiting.

Interrupt-Driven I/O

CPU initiates I/O and continues with other tasks. The device interrupts the CPU when it has completed the I/O operation. CPU suspends current task, executes Interrupt Service Routine (ISR), then resumes. More efficient than polling.

Direct Memory Access (DMA)

A specialized DMA controller handles data transfer directly between I/O device and main memory WITHOUT CPU involvement. CPU just initializes the transfer (source, destination, count) and is free to do other work. DMA interrupts CPU only when the entire transfer is complete. Ideal for high-speed, large data transfers (disk, network).

3.3 Asynchronous Data Transfer

In asynchronous communication, sender and receiver have no shared clock. Timing is negotiated through control signals.

Handshaking

A protocol for synchronizing data transfer: (1) Sender asserts "data ready" signal. (2) Receiver processes data and asserts "data received" (acknowledge). (3) Sender deasserts "data ready". (4) Receiver deasserts acknowledge. This ensures reliable transfer without data loss.

3.4 Serial vs Parallel Communication

- **Serial Communication:** One bit at a time over a single wire. Slower but uses fewer wires; suitable for long distances. Examples: UART, USB, RS-232.
 - **Parallel Communication:** Multiple bits simultaneously over multiple wires. Faster but requires more wires; suitable for short distances. Examples: Parallel port (LPT).
 - **Character-Oriented Protocol:** Data transmitted as characters (bytes). Uses start/stop bits for framing.
 - **Bit-Oriented Protocol:** Data transmitted as bit streams. More efficient for large data blocks.
-

UNIT 4: Memory Organization

4.1 Memory Hierarchy

Memory is organized in a hierarchy based on speed, cost, and capacity. From fastest/most expensive to slowest/cheapest: Registers -> Cache Memory -> Main Memory (RAM) -> Auxiliary Storage (HDD/SSD) -> Optical/Tape Storage. The goal: provide the illusion of fast, large, affordable memory.

4.2 Types of Memory

- **Registers:** Fastest, inside CPU. Very small (few dozen to hundreds). Used for immediate calculations.
- **Cache Memory (SRAM):** Very fast, stores frequently accessed data. L1 (smallest, fastest), L2, L3 caches.
- **RAM (Random Access Memory):** Main memory. DRAM — slower, cheaper, denser. SDRAM, DDR SDRAM — synchronous types.
- **ROM (Read-Only Memory):** Non-volatile; stores firmware/BIOS. Types: ROM, PROM, EPROM, EEPROM, Flash.
- **Auxiliary Memory:** Magnetic Disk (HDD), SSD, Magnetic Tape — large capacity, non-volatile, slow.

4.3 Associative Memory (Content-Addressable Memory)

Associative memory is accessed by the CONTENT of the data rather than by address. Search key is compared simultaneously with all stored words. Returns the matching word(s). Used in cache memory for tag lookup and TLBs (Translation Lookaside Buffers).

4.4 Cache Memory

Cache is a small, fast memory between the CPU and main RAM. It exploits the principle of locality: Temporal locality — recently used data will be used again soon. Spatial locality — data near recently accessed data will soon be accessed.

Cache Mapping Techniques

- **Direct Mapping:** Each main memory block maps to exactly one cache line. Simple and fast but high conflict miss rate.
- **Fully Associative Mapping:** Any memory block can go into any cache line. Best hit rate but complex and expensive comparator logic.
- **Set-Associative Mapping:** Cache divided into sets; each set has k lines (k-way). A block maps to one set, can go into any line within that set. Best compromise between direct and fully associative.

Performance Metrics

Hit Ratio (h) = Cache hits / Total memory accesses. Miss Ratio = 1 - h. Effective Memory Access Time (EMAT) = $h * T_c + (1-h) * T_m$ where T_c = cache access time, T_m = main memory access time.

4.5 Virtual Memory

Virtual memory is a memory management technique that gives programs the illusion of having more memory than physically available. Programs use virtual addresses; the OS maps them to physical addresses.

Address Space: Set of all virtual addresses. Memory Space: Set of physical memory locations. Mapping done via page tables in OS.

Page Replacement Algorithms

- **FIFO (First In First Out):** Replaces the page that has been in memory the longest. Simple but can suffer from Belady's anomaly.
- **LRU (Least Recently Used):** Replaces the page that hasn't been used for the longest time. Better performance than FIFO; approximates optimal strategy.

UNIT 5: Pipelining and Vector Processing

5.1 Introduction to Pipelining

Pipelining is a technique where multiple instructions are overlapped in execution. Like an assembly line — while one instruction is being executed, the next is being decoded, and the one after is being fetched. This improves throughput (instructions completed per unit time).

5.2 Pipeline Stages

A typical 5-stage instruction pipeline:

- **IF — Instruction Fetch:** Read instruction from instruction memory using PC.

- **ID — Instruction Decode:** Decode the instruction; read register values.
- **EX — Execute:** ALU performs arithmetic/logic operation or calculates address.
- **MEM — Memory Access:** Read from or write to data memory (for load/store instructions).
- **WB — Write Back:** Write result to destination register.

With an ideal pipeline, throughput = 1 instruction per clock cycle (after the pipeline is filled).

5.3 Pipeline Hazards

Structural Hazards

Occur when two instructions need the same hardware resource at the same time. Example: Single memory unit needed for both fetch and data access simultaneously. Solution: Separate instruction and data memories, or pipeline stalls.

Data Hazards

Occur when an instruction depends on the result of a previous instruction not yet completed. Types: RAW (Read After Write — true dependency), WAR (Write After Read), WAW (Write After Write). Solutions: Forwarding/bypassing (feed result directly to where needed), Pipeline stalls (NOP insertion).

Control Hazards (Branch Hazards)

Occur when a branch instruction changes the PC and instructions already in the pipeline are from the wrong path. Solutions: Branch prediction, Delayed branching, Branch target buffer.

5.4 Performance Metrics

Access Time

The time between initiating a memory request and the data being available. Key factor in memory system performance.

Effective Memory Access Time (EMAT)

$EMAT = h * T_c + (1-h) * (T_c + T_m)$ where h = hit ratio, T_c = cache time, T_m = main memory time.
Simplified: $EMAT = h * T_c + (1-h) * T_m$ when cache miss means main memory only.

Speedup with Pipelining

Ideal speedup = Number of pipeline stages. Actual speedup is less due to hazards, pipeline fill/drain time.
 $Speedup = \text{Time without pipeline} / \text{Time with pipeline}.$
